

PatientCareApi Project Report

PatientCareApi Project Report For A Junior Developer

1. Project Overview

PatientCareApi is a .NET ASP.NET Core Web API project for managing patients, treatments, and appointments.

The project is designed as an interview practice backend system. The goal is not only to make endpoints work, but also to show clean architecture, GitHub workflow, dependency injection, validation, EF Core, error handling, and good documentation.

The API is built feature by feature. Each feature is developed on its own branch, explained in documentation, and pushed through a pull request.

2. User Story

As a clinic staff member, I want to manage patient records, treatments, and appointments so that I can keep healthcare information organized and accessible through a REST API.

3. Main Business Requirements

- * Create, read, update, and delete patients.
- * Add treatments for an existing patient.
- * View all treatments or treatments for one patient.
- * Create appointments for an existing patient.
- * View all appointments or appointments for one patient.
- * Complete an appointment.
- * Cancel an appointment.
- * Return proper HTTP status codes.
- * Validate incoming request data.
- * Store data in SQLite using EF Core.
- * Keep controllers thin.
- * Keep business logic outside controllers.
- * Use clean layered architecture.

4. Main API Resources

The project has three main resources:

- * Patient
- * Treatment
- * Appointment

Patient is the central resource. Treatments and appointments belong to a patient.

5. Domain Model

Patient

A patient represents a person receiving care.

Important properties:

- * Id
- * FirstName
- * LastName
- * DateOfBirth
- * Email

- * PhoneNumber
- * CreatedAt
- * UpdatedAt
- * Treatments
- * Appointments

Important behavior:

- * Update patient details through a domain method.
- * Validate required names.
- * Validate date of birth is in the past.

Why this matters:

Domain entities should protect their own important state. Instead of allowing any code to freely set values, important changes happen through methods.

Treatment

A treatment represents care or medical work done for a patient.

Important properties:

- * Id
- * PatientId
- * Name
- * Description
- * TreatmentDate
- * Notes
- * CreatedAt

Important rules:

- * Treatment must belong to an existing patient.
- * Treatment name is required.

Appointment

An appointment represents a scheduled visit or meeting with a patient.

Important properties:

- * Id
- * PatientId
- * AppointmentDate
- * Reason
- * Status
- * CreatedAt
- * UpdatedAt

Important behavior:

- * Complete()
- * Cancel()

Why this matters:

Appointment status changes are business rules. They belong in the domain entity, not in the controller.

AppointmentStatus

AppointmentStatus is an enum:

- * Scheduled
- * Completed
- * Cancelled

Enums make status values safer than plain strings because the code can only use known valid values.

6. Clean Architecture Design

The solution uses five projects:

- * PatientCareApi.Domain
- * PatientCareApi.Application
- * PatientCareApi.Infrastructure
- * PatientCareApi.Api
- * PatientCareApi.Tests

The dependency direction is:

Api -> Application -> Domain

Api -> Infrastructure -> Application -> Domain

Tests -> projects needed for testing

The important rule:

Domain does not depend on any other project.

7. Domain Layer

The Domain layer contains the core business objects and behavior.

It contains:

- * Patient
- * Treatment
- * Appointment
- * AppointmentStatus

The Domain layer does not know about:

- * Controllers
- * HTTP
- * EF Core
- * SQLite
- * Swagger

Why this is good:

The core business model stays independent. If the database or API framework changes, the domain model can stay mostly the same.

8. Application Layer

The Application layer contains use-case focused code.

It contains:

- * DTOs
- * Repository interfaces
- * Service interfaces
- * Service implementations
- * Mapping helpers
- * Validation helper

- * NotFoundException
- * IUnitOfWork

The Application layer coordinates workflows.

Example workflow:

Create patient:

1. Validate request DTO.
2. Create Patient domain entity.
3. Add patient through IPatientRepository.
4. Save through IUnitOfWork.
5. Return PatientResponse DTO.

The Application layer depends only on Domain.

9. DTOs

DTO means Data Transfer Object.

DTOs define what data comes into the API and what data goes out of the API.

Examples:

- * CreatePatientRequest
- * UpdatePatientRequest
- * PatientResponse
- * CreateTreatmentRequest
- * TreatmentResponse
- * CreateAppointmentRequest
- * AppointmentResponse

Why DTOs are used:

- * They prevent exposing domain or database entities directly.
- * They let the API control its input and output shape.
- * They make validation clearer.
- * They protect the internal model from external clients.

10. Service Layer

Services contain application workflows.

Important services:

- * PatientService
- * TreatmentService
- * AppointmentService

Services are responsible for:

- * Validating requests.
- * Checking if related records exist.
- * Calling repository interfaces.
- * Calling domain methods.
- * Saving through Unit of Work.
- * Returning response DTOs.

Controllers should not contain business logic. Controllers call services.

11. Repository Interfaces

Repository interfaces live in the Application layer.

Examples:

- * IPatientRepository
- * ITreatmentRepository
- * IAppointmentRepository

They describe what data operations the Application layer needs.

They do not use EF Core directly.

Why this matters:

Application code depends on abstractions, not database implementation details.

12. Unit Of Work

IUnitOfWork represents saving or committing all pending changes.

The service does this:

```
await _patientRepository.AddAsync(patient, cancellationTokens);  
await _unitOfWork.SaveChangesAsync(cancellationTokens);
```

Repository stages the change. Unit of Work commits it.

Why this is useful:

If one service method uses multiple repositories, it can save once at the end.

Interview explanation:

Repositories handle data access operations. Unit of Work handles committing changes.
This keeps saving as one responsibility.

13. Infrastructure Layer

The Infrastructure layer contains EF Core and SQLite details.

It contains:

- * PatientCareDbContext
- * PatientRepository
- * TreatmentRepository
- * AppointmentRepository
- * UnitOfWork

Infrastructure depends on Application and Domain.

This means Infrastructure can implement Application interfaces.

14. EF Core DbContext

PatientCareDbContext represents the database session.

It contains:

- * DbSet<Patient>
- * DbSet<Treatment>
- * DbSet<Appointment>

It also configures:

- * Primary keys
- * Required fields
- * Max lengths

- * Relationships
- * Cascade delete
- * AppointmentStatus conversion to string
- * Backing-field collection access for Patient relationships

Why DbContext is in Infrastructure:

EF Core is a persistence detail. It should not be inside Domain or Application.

15. Repository Implementations

Repository implementations live in Infrastructure.

PatientRepository

PatientRepository handles patient database operations.

Important logic:

- * GetAllAsync uses AsNoTracking for read-only performance.
- * GetByIdAsync returns a tracked entity because update and delete workflows need EF Core to detect changes.
- * AddAsync stages a new patient.
- * DeleteAsync marks a patient for deletion.

TreatmentRepository

TreatmentRepository handles treatment database operations.

Important logic:

- * It filters treatments by PatientId in the database.
- * It uses AsNoTracking for read-only queries.
- * It stages new treatments with AddAsync.

AppointmentRepository

AppointmentRepository handles appointment database operations.

Important logic:

- * Read-only list queries use AsNoTracking.
- * GetByIdAsync returns a tracked appointment so AppointmentService can call Complete or Cancel and then save.

UnitOfWork

UnitOfWork calls DbContext.SaveChangesAsync.

This is the one place where the Application workflow is committed to the database.

16. API Layer

The API layer exposes HTTP endpoints.

It contains:

- * Program.cs
- * Controllers
- * appsettings.json
- * Swagger setup
- * ErrorHandlingMiddleware

The API layer depends on Application and Infrastructure.

17. Program.cs

Program.cs is the application startup file.

It does three main things:

1. Registers services in dependency injection.
2. Builds the application.
3. Configures middleware and routes.

Important registrations:

- * AddControllers
- * AddOpenApi
- * AddSwaggerGen
- * AddDbContext
- * Repository interfaces to repository implementations
- * Service interfaces to service implementations
- * IUnitOfWork to UnitOfWork

Program.cs is also called the composition root.

Composition root means:

This is where the outer application connects interfaces to concrete classes.

18. Dependency Injection

Dependency Injection means classes receive their dependencies instead of creating them directly.

Example:

PatientsController receives IPatientService.

PatientService receives IPatientRepository and IUnitOfWork.

Why this is good:

- * Classes are loosely coupled.
- * Code is easier to test.
- * Implementation can change without changing the caller.

19. Controllers

Controllers handle HTTP concerns.

Current controllers:

- * PatientsController
- * TreatmentsController
- * AppointmentsController

Controllers are responsible for:

- * Route definitions
- * Accepting request DTOs
- * Calling services
- * Returning HTTP responses

Controllers are not responsible for:

- * EF Core queries
- * Business rules

- * Manual validation logic
- * Database saving

20. Main Endpoints

Patients:

- * GET /api/patients
- * GET /api/patients/{id}
- * POST /api/patients
- * PUT /api/patients/{id}
- * DELETE /api/patients/{id}

Treatments:

- * GET /api/treatments
- * GET /api/patients/{id}/treatments
- * POST /api/patients/{id}/treatments

Appointments:

- * GET /api/appointments
- * GET /api/patients/{id}/appointments
- * POST /api/appointments
- * PATCH /api/appointments/{id}/complete
- * PATCH /api/appointments/{id}/cancel

21. Error Handling

The API uses `ErrorHandlingMiddleware`.

It maps exceptions to HTTP status codes:

- * `NotFoundException` -> 404 Not Found
- * `ValidationException` -> 400 Bad Request
- * `ArgumentException` -> 400 Bad Request
- * Unexpected exception -> 500 Internal Server Error

Why middleware is used:

It avoids repeated try/catch blocks in every controller.

Example response:

```
{
  "statusCode": 404,
  "message": "Patient with id 999 was not found."
}
```

22. Validation

Validation happens in multiple places:

- * DTO validation attributes
- * `RequestValidator` helper
- * Domain constructors and methods

Why multiple levels:

DTO validation checks incoming API data.

Application validation checks workflow rules.

Domain validation protects the core entity from invalid state.

23. Swagger

Swagger/OpenAPI gives an interactive API documentation page.

It helps developers:

- * See all endpoints.
- * Try requests.
- * Inspect request and response models.
- * Test the API without Postman.

Swagger is enabled in development.

24. SQLite

SQLite is used as a lightweight database.

Why SQLite is good for this project:

- * Simple setup.
- * Good for interview practice.
- * No separate database server needed.
- * Works well with EF Core.

The connection string is stored in appsettings.json:

Data Source=patientcare.db

25. GitHub Workflow

The project is built feature by feature.

Branch strategy:

- * main = stable final code
- * development = integration branch
- * feature/* = one feature at a time

Typical feature flow:

1. Start from development.
2. Create a feature branch.
3. Code the feature.
4. Run build/tests.
5. Commit.
6. Push branch.
7. Open pull request into development.
8. Resolve conflicts if needed.
9. Merge PR.

26. Feature Timeline

Completed features:

- * Feature 01: Solution structure and Domain model
- * Feature 02: DTOs, repository interfaces, service interfaces, Unit of Work contract
- * Feature 03: Application services, validation, mapping, NotFoundException
- * Feature 04: EF Core DbContext, repository implementations, UnitOfWork implementation
- * Feature 05: API dependency injection, Swagger, controllers
- * Feature 06: Global error handling middleware

Remaining features:

- * Feature 07: EF Core migrations and database creation
- * Feature 08: Unit tests
- * Feature 09: GitHub Actions CI pipeline
- * Feature 10: Dockerfile and Docker image workflow
- * Feature 11: Final documentation cleanup and interview review

27. How To Run The API

Restore packages:

```
dotnet restore --configfile NuGet.Config
```

Build:

```
dotnet build --no-restore
```

Run:

```
dotnet run --project PatientCareApi.Api
```

Open Swagger:

```
https://localhost:<port>/swagger
```

28. How To Explain The Project In An Interview

You can say:

I built a healthcare REST API using ASP.NET Core and clean architecture. The Domain layer contains the core business entities and rules. The Application layer contains DTOs, service logic, repository interfaces, validation, mapping, and Unit of Work. The Infrastructure layer implements persistence with EF Core and SQLite. The API layer exposes REST endpoints through thin controllers, registers dependencies in Program.cs, provides Swagger, and uses global error handling middleware.

29. Key Learning Points

- * Keep business logic out of controllers.
- * Use DTOs for API input and output.
- * Use services for workflows.
- * Use repository interfaces to hide database details.
- * Use Infrastructure to implement EF Core persistence.
- * Use Unit of Work to save changes.
- * Use global middleware for consistent error responses.
- * Use GitHub feature branches and pull requests.
- * Keep documentation updated after every feature.

30. Final Advice For A Junior Developer

Do not try to understand everything at once.

Start with the flow of one request:

```
POST /api/patients
```

Then follow the code:

```
PatientsController -> IPatientService -> PatientService -> IPatientRepository ->  
PatientRepository -> PatientCareDbContext -> SQLite
```

That request flow explains most of the architecture.